

PocketSmart AI: Smart Budget & Recommendation Assistant using Gemini Models

Project Description

PocketSmart AI is a web application that helps users plan a budget for a home interior, a party or a jewelry purchase. A FastAPI backend authenticates the user, validates the budget and requirements, and sends a structured prompt to Google Gemini. The response is turned into a category-wise plan in INR with a calculation table, the remaining budget and shopping search links for Indian platforms. If Gemini is unavailable, a rule-based fallback estimate is shown. Every result is saved in the user's history during the session.

Scenarios

Scenario 1: Home Interior Plan. The user enters a total budget, how many lights, fans, furniture pieces and dining tables are needed, and which rooms to furnish. Gemini returns a breakdown by category with product suggestions, estimated prices and search links for Amazon, Flipkart, IKEA, Myntra and Ajio.

Scenario 2: Party Plan. The user enters the budget, number of guests, party type, venue type and whether catering, decoration and entertainment are needed. The plan covers items such as venue, food, decoration and entertainment, with links to platforms such as Swiggy, Zomato, BookMyShow, Booking.com and MakeMyTrip.

Scenario 3: Jewelry Plan. The user enters a budget, an occasion and style preferences and may upload a photo of the outfit. Gemini analyses the outfit colours, style and formality and suggests matching jewelry with styling tips, with links to BlueStone, Tanishq, CaratLane, Melorra and others.

Scenario 4: History and Fallback. Every plan is saved to the user's history and can be reopened. If the AI service fails, the page shows a notice and a standard budget split instead of an error.

Technical Architecture

Three layers: Jinja2 templates with shared JavaScript in the browser; a FastAPI application (main.py, auth.py, models.py); and an AI layer (gemini_utils.py) that calls Google Gemini. User accounts, sessions and history are held in memory and uploaded images are stored in static/uploads/. See 3. Project Design Phase / Solution Architecture for the diagram.

Pre-requisites

1. Python 3.10+ – <https://www.python.org>
2. FastAPI – <https://fastapi.tiangolo.com>
3. Google AI Studio account and Gemini API key – <https://aistudio.google.com/apikey>
4. Jinja2 templates – <https://jinja.palletsprojects.com>
5. Uvicorn ASGI server – <https://www.uvicorn.org>
6. pytest for running the tests – <https://docs.pytest.org>
7. Git and a GitHub account
8. Basic HTML, CSS and JavaScript

Project Workflow

Milestone	Activities	Where in the code
1. Model Selection & Architecture	Choose Gemini models; define the architecture; set up the environment	requirements.txt, .env.example, main.py
2. Core Functionalities	Home, party and jewelry planners; fallback estimates; history	gemini_utils.py, models.py
3. Routing and Authentication	Page routes, planner endpoints, sign-in, sessions	main.py, auth.py
4. Frontend Development	Design pages; dynamic results with JavaScript	templates/, static/
5. Testing & Local Deployment	Run pytest; verify local deployment	tests/test_app.py

Milestone 1: Model Selection and Architecture

1. Review the needs: structured budget plans in INR, shopping links, image-aware jewelry suggestions and fast responses.
2. Choose Google Gemini because it accepts both text and images through a simple API and needs no local model hosting.
3. Configure a list of lightweight Gemini models (gemini-3.5-flash-lite, gemini-2.5-flash-lite, gemini-flash-lite-latest) that are tried in order.
4. Create a virtual environment and install the libraries from requirements.txt.
5. Run the server and open <http://127.0.0.1:8000> (API docs at /docs).

Model names change over time. Check Google AI Studio for the models available to your key and update MODEL_CANDIDATES in gemini_utils.py if needed.

Milestone 2 and 3: Backend

- models.py defines RegisterUser, HomeBudgetInput, PartyBudgetInput, JewelryBudgetInput and the session and history records, with validators.
- gemini_utils.py builds the prompts, calls Gemini with retry and model fallback, extracts the JSON, adds shopping links, saves the history and returns static fallbacks.
- auth.py handles bcrypt password hashing, JWT creation and verification, sessions and the token blacklist.
- main.py exposes: /, /login, /register, /dashboard, /home-planner, /party-planner, /jewelry-planner, /history, POST /register, POST /token, POST /logout, /session-info, /session-data, POST /home-budget, /party-budget, /jewelry-budget, GET /recommendation-history and /recommendation-details/{id}.

Milestone 4: Frontend Development

Page	Route	Purpose
Landing page	/	Introduces the planners and links to sign-in and registration
Create Account	/register	Registration form
Sign In	/login	Login form

Dashboard	/dashboard	Planner cards and recent activity
Home Interior Planner	/home-planner	Form and results for the home plan
Party Planner	/party-planner	Form and results for the party plan
Jewelry Planner	/jewelry-planner	Form, outfit upload and results
History	/history	List and details of past recommendations

Pages share `_head.html` and `_navbar_app.html`, use `styles.css` for the layout, and call the API through the helpers in `static/js/app.js`.

Milestone 5: Testing and Local Deployment

Run pytest tests to execute the 11 automated tests. Then start the server, register a user and test each planner page, including a run without a Gemini key to see the fallback notice.

Screenshots

Add the application screenshots here: landing page, sign-in, dashboard, each planner with a generated result, the fallback notice and the history page.

Conclusion

PocketSmart AI combines FastAPI with Google Gemini to turn a budget and a few requirements into an itemised plan in INR with shopping links. Validation, model fallback, a labelled static estimate and per-user history make it dependable for a local prototype, and the clear module structure leaves room to add persistent storage and more planners.